

Docker for Machine Learning and AI Engineering

1. Overview

Docker packages an application, its runtime, system libraries, language dependencies, configuration defaults, and startup command into a versioned **image**. A **container** is a running instance of that image.

For ML, the deployable application is more than `model.predict(x)`: it includes the Python version, preprocessing, model artifact, compiled numerical libraries, CUDA user-space libraries, HTTP server, and configuration. Docker gives that whole unit a repeatable build and runtime interface.

```
code + dependency lock + model + Dockerfile
      -> image -> tested image digest
      -> container on laptop/CI/cloud
```

Real uses include model-serving APIs, reproducible training jobs, batch inference, data-processing workers, experiment environments, and local multi-service stacks. Docker is not a training framework, registry for ML metadata, workflow scheduler, or multi-host orchestrator; tools such as PyTorch, MLflow, Airflow, and Kubernetes solve those adjacent problems.

2. Intuition

Sending only an ML script is like sending an experiment protocol without the equipment. An image is a sealed, versioned experiment kit; a container is one use of that kit.

Term	Analogy	Reality
Dockerfile	Recipe	Build instructions
Image	Template	Read-only layers and runtime metadata
Container	Instance	Isolated process plus writable layer
Registry	Warehouse	Stores and distributes images
Volume	External drive	Persistent data outside container lifecycle
Compose	Blueprint	Defines cooperating services

An image can create many containers. Removing a container removes its writable layer, not the image or named volumes. Containers are not lightweight VMs: they share the host kernel, whereas VMs normally boot separate guest kernels.

3. Prerequisites

- Command-line basics: files, processes, ports, environment variables, signals.
- Python packaging: `pip`, requirements/lock files, imports, virtual environments.
- HTTP and client-server basics.

- YAML for Compose and basic Linux commands.
- ML inference flow: validate, preprocess, infer, postprocess.
- Helpful: Git, CI/CD, registries, CPU/RAM/GPU concepts, CUDA compatibility, Kubernetes probes.

For interviews, know that Linux **namespaces** isolate views of processes, networks, users, and mounts, while **cgroups** account for and limit CPU and memory.

4. Core Concepts

4.1 Client and engine

The **docker** CLI sends API requests to Docker Engine. The daemon/builder manages images, containers, networks, and volumes. **docker ps** therefore queries the daemon; builder architecture, GPUs, paths, and cache belong to that host.

Interview angle: a remote client and daemon need not share a filesystem or CPU architecture.

4.2 Dockerfile instructions

Instruction	Meaning	Interview detail
FROM	Base image/build stage	First normal instruction; stages may have names
WORKDIR	Working directory	Prefer it to repeated cd
COPY	Copy from build context	Inputs affect build cache
RUN	Execute during build	Produces build result/layer
ENV	Runtime/build default	Persists in image metadata
ARG	Build-time value	Not a safe secret mechanism
EXPOSE	Document container port	Does not publish it
USER	Runtime user	Prefer non-root
ENTRYPOINT	Main executable	Harder to replace
CMD	Default command/arguments	Runtime arguments replace it

Use exec form—**CMD** ["**uvicorn**", ...]—for predictable argument and signal handling. **RUN** is build-time; **CMD** is runtime.

4.3 Images, layers, and copy-on-write

An image is content-addressed layers plus metadata. Conceptually:

$$F_{image} = L_1 \oplus L_2 \oplus \dots \oplus L_n, \quad F_{container} = F_{image} \oplus W$$

Later layers overlay earlier ones and W is container-writable state. Identical image layers are shared. Deleting a file in a later layer hides it but may not remove bytes from an earlier layer; avoid creating large unwanted files in the first place.

Interview angle: copy dependency manifests and install dependencies before copying frequently changing source so expensive layers remain cached.

4.4 Build context and `.dockerignore`

The build context is the file set available to `COPY/ADD`. ML repos often contain datasets, checkpoints, `.git`, notebooks, `.env`, and virtual environments. Exclude them to improve speed, cache stability, and security.

```
.git
.venv
.env
__pycache__/
data/
checkpoints/
```

`.gitignore` does not replace `.dockerignore`.

4.5 Container lifecycle

A container is tied to its main process, PID 1. When that process exits, the container stops. The process must remain in the foreground and handle SIGTERM for graceful shutdown. `docker run` creates a new container; `docker start` restarts an existing stopped one.

4.6 Isolation and limits

Namespaces isolate views of resources; cgroups enforce/account for resources. Example:

```
docker run --memory=2g --cpus=1.5 ml-api:1.0
```

A limit is not a reservation. Exceeding memory can lead to an OOM kill. Containers share a kernel, so non-root users, minimal capabilities, patched images, and restricted mounts/network remain necessary.

4.7 Networking

```
docker run --rm -p 8080:8000 ml-api:1.0
```

This maps host port 8080 to container port 8000. The server must bind to `0.0.0.0` inside the container. `EXPOSE 8000` is documentation, not publication. In Compose, `redis:6379` reaches the `redis` service; `localhost:6379` means the current container.

4.8 Storage

Type	Best use	Caution
Writable layer	Temporary output	Removed with container

Type	Best use	Caution
Bind mount	Host source/dataset	Host paths and permissions couple environments
Named volume	Persistent service data	Requires its own backup lifecycle
<code>tmpfs</code>	Fast/sensitive temporary data	Uses RAM and disappears on stop

Large datasets should normally be mounted read-only or read from object storage, not copied into every image.

4.9 Registry, tag, and digest

`registry.example.com/ml/churn-api:1.4.2` contains registry, repository, and tag. Tags—including `latest`—are mutable pointers. A digest such as `image@sha256:...` identifies immutable content. Promote the tested digest for reliable rollbacks.

4.10 Configuration and secrets

Use runtime environment variables or mounted config for environment-specific values. Never bake credentials into an image or commit `.env`. Build arguments and environment variables are inappropriate for build secrets; BuildKit secret mounts expose a credential only to the relevant build step.

4.11 Compose

Compose describes local multi-container applications in YAML, including networks, volumes, environment, health checks, and dependency order. It is excellent for development and integration tests. It is not a multi-host scheduler with autoscaling and rollout control.

4.12 Cache and multi-stage builds

BuildKit reuses steps whose instruction and relevant inputs match. Once an input changes, dependent later work rebuilds. Cache mounts preserve package downloads without copying cache into the final image.

Multi-stage builds use several `FROM` stages: compile wheels or binaries in a builder and copy only runtime artifacts into a smaller final image. This reduces size and attack surface, but reproducibility still requires pinned dependencies and base-image identity.

4.13 Health and readiness

- **Liveness:** should the process be restarted?
- **Readiness:** can it receive traffic now?
- **Startup:** did slow model initialization complete?

Docker supplies a general `HEALTHCHECK`; orchestrators may separate these probes. A large model can leave the process alive but not ready. Probes must be cheap—expensive inference probes can become load.

4.14 Architecture and GPU

Images target platforms such as `linux/amd64` and `linux/arm64`. Cross-platform builds can use `buildx`, but emulation may be slow. GPU images contain user-space CUDA/framework libraries; the compatible NVIDIA

host driver and GPU remain outside the container. `--gpus all` exposes configured GPUs—it does not create one.

5. Algorithm / Working Process

Build

1. Select the context and apply `.dockerignore`.
2. Parse Dockerfile stages and arguments.
3. Resolve/pull the `FROM` image for the target platform.
4. Check cache for each instruction.
5. Execute cache misses and record content-addressed results.
6. Produce image configuration, manifest, digest, and optional tag.

Run

1. Resolve/pull the image.
2. Create a writable layer, namespaces, cgroups, mounts, and networking.
3. combine `ENTRYPOINT`, `CMD`, and command-line overrides.
4. Start PID 1 with the configured user/environment.
5. Load preprocessing and the model once.
6. Bind the server, become ready, and process requests.
7. Log to stdout/stderr.
8. On stop, receive SIGTERM, drain, and exit before forced termination.

Inference path

```
JSON -> schema validation -> feature ordering -> preprocessing
      -> predict/predict_proba -> postprocessing -> versioned JSON response
```

Training and serving usually use different images or commands because their dependencies, privileges, scaling, and hardware differ. A release pipeline should build, test, scan, push, deploy the same immutable digest, health-check it, monitor it, and retain rollback.

6. Mathematical Foundation

Docker has no loss function or optimizer. Relevant mathematics is operational.

Storage

For image layer sets L_j and compressed layer sizes s_i , deduplicated storage is:

$$S_{unique} = \sum_{i \in \cup_j L_j} s_i, \quad S_{total} \approx S_{unique} + \sum_{k=1}^C w_k$$

where w_k is each container's writable data.

Build caching

With step cost t_i and cache-hit indicator $h_i \in \{0, 1\}$:

$$T_{build} \approx T_{context} + \sum_i (1 - h_i) t_i$$

Stable expensive instructions should precede frequently changing inputs.

Memory

For a worker:

$$M_{worker} \approx M_{runtime} + M_{model} + BM_{sample} + M_{workspace}$$

For W processes, conservatively budget WM_{worker} plus OS and safety margin. Some CPU pages may share copy-on-write, but GPU weights and mutated objects often do not.

For P parameters stored at b bits:

$$M_{weights} \approx \frac{Pb}{8} \text{ bytes}$$

A 7B-parameter model at 16 bits needs about 14 GB just for weights, before KV cache, activations, allocator fragmentation, and framework overhead.

Throughput and latency

With average service time S , ideal rate is $\mu \approx 1/S$. Little's Law gives:

$$L = \lambda T$$

At 20 requests/s and 0.25 s average total time, average in-flight concurrency is 5. End-to-end latency decomposes as:

$$T_{total} = T_{network} + T_{queue} + T_{pre} + T_{model} + T_{post}$$

As utilization $\rho = \lambda/\mu$ approaches 1, queueing latency can rise sharply; capacity needs headroom.

Cost and ML metrics

$$C_{total} \approx NHc_h + C_{storage} + C_{egress} + C_{build}, \quad C_{request} = C_{total}/N_{requests}$$

Model loss and quality metrics remain properties of the task. Docker adds operational metrics: p50/p95/p99 latency, throughput, errors, cold-start time, utilization, memory, restarts, OOMs, and image-pull time.

7. Practical Implementation

This end-to-end example trains a small scikit-learn pipeline during the image build and serves it with FastAPI. Real training should normally be a separate tracked pipeline; this build-time training is only for a deterministic, seconds-long demonstration.

Project layout

```
docker-ml-demo/  
├─ app/__init__.py  
├─ app/train.py  
├─ app/main.py  
├─ tests/smoke_test.py  
├─ requirements.txt  
├─ .dockerignore  
├─ Dockerfile  
└─ compose.yaml
```

`requirements.txt` (illustrative pinned versions):

```
fastapi==0.116.1  
joblib==1.5.1  
scikit-learn==1.7.1  
uvicorn==0.35.0
```

`app/train.py`:

```
from pathlib import Path  
import joblib  
from sklearn.datasets import load_iris  
from sklearn.linear_model import LogisticRegression  
from sklearn.pipeline import make_pipeline  
from sklearn.preprocessing import StandardScaler  
  
def train(path: str = "/app/artifacts/iris.joblib") -> None:  
    data = load_iris()  
    model = make_pipeline(  
        StandardScaler(),  
        LogisticRegression(max_iter=300, random_state=42),  
    )  
    model.fit(data.data, data.target)  
    destination = Path(path)  
    destination.parent.mkdir(parents=True, exist_ok=True)  
    joblib.dump(model, destination)  
  
if __name__ == "__main__":  
    train()
```

`app/main.py`:

```
import os  
from contextlib import asynccontextmanager
```

```
from pathlib import Path
from typing import Annotated

import joblib
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

MODEL_PATH = Path(os.getenv("MODEL_PATH", "/app/artifacts/iris.joblib"))
MODEL_VERSION = os.getenv("MODEL_VERSION", "iris-logreg-1")
model = None

class IrisFeatures(BaseModel):
    sepal_length: Annotated[float, Field(gt=0, le=20)]
    sepal_width: Annotated[float, Field(gt=0, le=20)]
    petal_length: Annotated[float, Field(gt=0, le=20)]
    petal_width: Annotated[float, Field(gt=0, le=20)]

@asynccontextmanager
async def lifespan(_: FastAPI):
    global model
    if not MODEL_PATH.is_file():
        raise RuntimeError(f"Missing model: {MODEL_PATH}")
    model = joblib.load(MODEL_PATH) # Load trusted artifacts only.
    yield
    model = None

api = FastAPI(title="Iris classifier", lifespan=lifespan)
CLASS_NAMES = ["setosa", "versicolor", "virginica"]

@api.get("/live")
def live():
    return {"status": "alive"}

@api.get("/ready")
def ready():
    if model is None:
        raise HTTPException(503, "Model is not loaded")
    return {"status": "ready", "model_version": MODEL_VERSION}

@api.post("/predict")
def predict(x: IrisFeatures):
    if model is None:
        raise HTTPException(503, "Model is not loaded")
    row = [[x.sepal_length, x.sepal_width, x.petal_length, x.petal_width]]
    probabilities = model.predict_proba(row)[0]
    class_id = int(probabilities.argmax())
    return {
        "class_id": class_id,
```

```

    "class_name": CLASS_NAMES[class_id],
    "probabilities": [round(float(p), 6) for p in probabilities],
    "model_version": MODEL_VERSION,
}

```

Dockerfile:

```

# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip wheel --wheel-dir=/wheels -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \
    MODEL_PATH=/app/artifacts/iris.joblib MODEL_VERSION=iris-logreg-1
WORKDIR /app
COPY --from=builder /wheels /wheels
RUN pip install --no-cache-dir /wheels/* && rm -rf /wheels \
    && useradd --create-home --uid 10001 appuser
COPY --chown=appuser:appuser app/ ./app/
RUN mkdir -p /app/artifacts && python -m app.train \
    && chown -R appuser:appuser /app/artifacts
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:api", "--host", "0.0.0.0", "--port", "8000"]

```

.dockerignore:

```

.git
.venv
.env
__pycache__/
*.py[cod]
data/
checkpoints/
notebooks/
tests/

```

Build, run, and call it:

```

docker build -t iris-api:1.0 .
docker run --rm --name iris-api -p 8000:8000 iris-api:1.0
curl http://localhost:8000/ready
curl -X POST http://localhost:8000/predict \

```

```
-H "Content-Type: application/json" \  
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

compose.yaml:

```
services:  
  api:  
    build: .  
    image: iris-api:1.0  
    ports: ["8000:8000"]  
    init: true  
    restart: unless-stopped  
    environment:  
      MODEL_VERSION: iris-logreg-1  
    healthcheck:  
      test: ["CMD", "python", "-c", "import urllib.request;  
urllib.request.urlopen('http://localhost:8000/ready', timeout=2)"]  
      interval: 10s  
      timeout: 3s  
      retries: 3  
      start_period: 20s
```

```
docker compose up --build -d  
docker compose ps  
docker compose logs -f api  
docker compose exec api python -c "import sklearn; print(sklearn.__version__)"  
docker compose down
```

Minimal tests/smoke_test.py:

```
import json  
from urllib.request import Request, urlopen  
  
with urlopen("http://localhost:8000/ready", timeout=5) as response:  
    assert response.status == 200  
  
body = json.dumps({"sepal_length": 5.1, "sepal_width": 3.5,  
                  "petal_length": 1.4, "petal_width": 0.2}).encode()  
request = Request("http://localhost:8000/predict", data=body,  
                 headers={"Content-Type": "application/json"}, method="POST")  
with urlopen(request, timeout=5) as response:  
    result = json.load(response)  
    assert result["class_name"] == "setosa"  
    assert abs(sum(result["probabilities"]) - 1) < 1e-5
```

For a large independently released model, mount a verified artifact read-only and set `MODEL_PATH`. This avoids rebuilding huge images, but deployment must atomically coordinate image version, model version, checksum, schema, and framework compatibility.

Build secret example:

```
RUN --mount=type=secret,id=pip_config,target=/etc/pip.conf \  
pip install private-ml-package==1.2.3
```

```
docker build --secret id=pip_config,src=/secure/pip.conf -t private-api:1.0 .
```

GPU smoke check (choose a real supported tag compatible with the host):

```
docker run --rm --gpus all nvidia/cuda:12.9.0-base-ubuntu22.04 nvidia-smi
```

8. Code Explanation

- `make_pipeline` serializes scaling with the classifier, preventing training-serving skew.
- `lifespan` loads the model once before readiness, instead of deserializing per request.
- Pydantic rejects missing, wrong-type, and out-of-contract fields before inference.
- Explicit feature order avoids valid-shaped but semantically wrong arrays.
- `model_version` makes responses traceable.
- Joblib/pickle-style artifacts can execute code and are version-sensitive; load only trusted, verified artifacts.
- The builder creates wheels; the runtime stage omits build state.
- Copying `requirements.txt` first preserves dependency cache when only source changes.
- UID 10001 reduces privilege; it is defense in depth, not a complete sandbox.
- `0.0.0.0` accepts traffic through the container interface.
- Exec-form `CMD` lets the server receive signals directly.
- `init: true` forwards signals and reaps orphaned children.
- The health check uses Python already installed rather than adding `curl` only for probing.

For production, pin a base digest, rebuild routinely for patches, scan, sign/attest if required, and promote the tested digest. Do not manually repair production with `docker exec`; change the source image and replace the container.

9. Training / Evaluation

Docker makes evaluation reproducible but does not replace it.

Model layer

- Split before learned preprocessing; use time/group splits where IID random split is invalid.

- Track feature schema, label mapping, data snapshot, code commit, dependencies, artifact checksum, and metrics.
- Choose task metrics: F1/PR-AUC for imbalance, log loss/calibration for probabilities, MAE/RMSE for regression, recall@k/NDCG for retrieval.
- Diagnose overfitting with train-validation gaps, learning curves, cross-validation, and slice metrics.

Image/container layer

Test that the image builds cleanly, starts without undeclared host state, becomes ready within budget, handles valid and invalid input, fails clearly on a corrupt artifact, shuts down on SIGTERM, runs non-root, respects limits, and passes vulnerability/secret policy.

Online layer

Concern	Metrics
Reliability	availability, 4xx/5xx, restarts, OOMs
Latency	p50/p95/p99, queue time, cold start
Capacity	requests/s, concurrency, batch utilization
Resources	CPU, RAM, GPU utilization/memory, I/O
ML quality	drift, confidence, calibration, delayed ground truth

Serving hyperparameters include workers, threads, batch size, timeout, request-size limit, CPU/RAM/GPU limits, and graceful-shutdown time. More workers may duplicate model memory; a large GPU model often benefits from one controlled model process with dynamic batching.

Reproducibility requires locked dependencies, base digest, explicit platform, versioned data/code/config/model, recorded image digest/SBOM, and promotion of the same tested image rather than rebuilds per environment.

10. Complexity and Cost

Image pull time is approximately:

$$T_{pull} \approx S_{missing,compressed}/B_{network} + T_{decompress} + T_{registry}$$

Runtime CPU is usually close to native because there is no full guest-OS emulation, but overlay writes, networking, logging, cgroup throttling, and cross-architecture emulation can matter. Measure the actual path.

Maximum worker count under a memory budget is roughly:

$$W_{max} \leq \left\lfloor \frac{M_{available} - M_{reserve}}{M_{worker}} \right\rfloor$$

Costs include CI build time, registry storage/egress, image pulls, warm replicas, logs, and—usually dominant for large models—GPU time. Optimize in this order: correctness/security, measured right-sizing, utilization/batching, cold start/pull size, then build speed.

11. Common Use Cases

1. **REST/gRPC inference:** deploy a validated model behind a stable interface.
2. **Batch inference:** run a versioned job over object-store or mounted data.
3. **Training jobs:** reproduce CUDA, framework, and system dependencies.
4. **CI tests:** test models and APIs in the release environment.
5. **Local platforms:** Compose an API, Redis, database, worker, and MLflow.
6. **Notebook environments:** provide consistent libraries with mounted user work.
7. **Edge packaging:** ship a platform-specific optimized runtime.
8. **Data processing:** isolate Spark/Python feature or validation jobs.
9. **Model-server extensions:** package custom Triton/TorchServe/vLLM logic.
10. **Interview/demo projects:** give reviewers one build/run workflow.

12. Common Mistakes

- Using `latest` in production and losing traceability.
- Copying the whole repo before dependency installation, destroying cache reuse.
- Including `.git`, credentials, datasets, or huge checkpoints accidentally.
- Running as root or granting `--privileged` without need.
- Storing database/model output only in the writable layer.
- Binding the server to `127.0.0.1` inside the container.
- Assuming `EXPOSE` publishes a port.
- Using `localhost` for a different Compose service.
- Confusing start order with readiness; dependencies also need runtime retry/backoff.
- Loading a model on every request or spawning too many memory-duplicating workers.
- Serializing preprocessing separately and changing its feature order.
- Loading untrusted pickle/joblib artifacts.
- Baking secrets into `ARG`, `ENV`, source, or image layers.
- Installing unpinned dependencies and expecting reproducibility.
- Using a full CUDA image for CPU inference.
- Solving drift, leakage, wrong metrics, or bad validation with Docker—containerization cannot fix model methodology.

13. Edge Cases / Limitations

- Containers share the host kernel and are not identical to VM isolation.
- Linux and Windows containers require compatible host/kernel arrangements.
- ARM/x86 mismatch may require multi-platform builds or slow emulation.
- GPU success depends on hardware, host driver, runtime toolkit, CUDA/framework compatibility, and device allocation.
- Very large images cause slow pulls, cold starts, and registry cost.
- Stateful workloads require explicit persistence, consistency, and backup design.
- Mutable external model mounts can create image/model version skew.
- Copy-on-write filesystems can be poor for write-heavy databases or training scratch I/O.
- Rootless/non-root modes may restrict ports, devices, or mounts.
- Air-gapped environments require mirrored images and dependencies.
- Docker alone does not provide multi-host scheduling, autoscaling, traffic shifting, or global secrets management.

- Bitwise reproducibility can still fail because of nondeterministic GPU kernels, floating-point order, unpinned data, or upstream artifacts.

14. Variations

Variation	What changes	When to use	Placement relevance
Multi-stage build	Separate build and runtime environments	Compiled wheels/binaries, smaller production images	High
Distroless/minimal runtime	Removes shell/package manager and extra tools	Hardened, well-understood production service	Medium; debugging trade-off matters
Rootless Docker	Daemon/containers avoid host root	Developer or server hardening where supported	Medium
Multi-platform image	Manifest points to ARM64 and AMD64 variants	Mixed laptop/cloud/edge targets	Medium-high
CPU vs CUDA image	Different base/runtime libraries	Match serving hardware exactly	High for ML roles
Image-bundled model	Weights copied into image	Smaller models, atomic image+model release	High
External/mounted model	Weights released separately	Huge or frequently updated weights	High; discuss coordination risk
Development container	Bind-mounted source and debug tools	Fast local iteration	Practical
Immutable production image	No live source mount; digest deployment	CI/CD and rollback	Essential
Compose	Several services on one host	Local stacks and integration tests	High
Kubernetes/container service	Scheduler manages replicated containers	Multi-host production, autoscaling, rollouts	High
Specialized model server	Triton, TorchServe, TF Serving, vLLM, TGI	Batching, GPU scheduling, optimized inference	High for AI engineering

Research: exact CUDA/framework environments and multi-stage builds aid artifact reproducibility. **Projects:** Compose and image-bundled small models give the simplest reviewer experience. **Production:** immutable digests, non-root operation, external secret stores, scanning, and orchestration matter most.

15. Related Topics

Docker vs virtual machines

Containers share the host kernel and start as processes; VMs virtualize hardware and boot guest kernels. Containers are generally smaller and faster to start. VMs provide a stronger boundary and can run a different guest kernel. They are often combined: containers run inside cloud VMs.

Docker vs Python virtual environments

A venv isolates Python packages only. Docker also specifies OS libraries, filesystem, user, network metadata, and startup command. Use a venv for lightweight Python development; use Docker when system-level repeatability and deployment packaging matter.

Docker vs Conda

Conda manages language/native packages and environments; Docker packages the operating environment and process. A Conda environment can live inside an image when its dependency solver/channels are genuinely required, but layering both increases size and complexity.

Docker Compose vs Kubernetes

Compose describes a multi-container application primarily for a single Docker environment. Kubernetes schedules containers across nodes and provides deployments, services, readiness/liveness/startup probes, rollouts, autoscaling, policies, and reconciliation. A Compose file is not automatically a production Kubernetes design.

Docker vs MLflow

Docker versions runtime environments. MLflow tracks experiments, parameters, metrics, artifacts, and model lifecycle. An MLflow model may be served inside Docker; neither replaces the other.

Docker vs model formats

ONNX, SavedModel, TorchScript/exported programs, and safetensors represent models or weights. Docker packages the runtime that loads them. Safer/non-Python formats can reduce arbitrary-code and compatibility risks, but format conversion must be numerically validated.

Docker vs CI/CD

Docker is a build/runtime unit. CI/CD automates testing, scanning, registry publication, approval, deployment, and rollback. A Dockerfile without pipeline checks is not a delivery strategy.

Docker and Kubernetes resource settings

Docker flags can limit a container on one host. Kubernetes distinguishes resource requests used by scheduling from limits enforced at runtime. Correct capacity planning requires both model measurements and orchestrator semantics.

16. Interview Questions

1. What is the difference between an image and a container?

An image is an immutable, content-addressed template containing filesystem layers and configuration. A container is a runtime instance with isolated process/network/mount state and a writable layer. Many

containers can share one image.

2. Why are containers lighter than VMs?

They normally share the host kernel and start ordinary isolated processes instead of booting a guest OS. This reduces startup time and per-instance memory/disk, but it also means the isolation and kernel compatibility model differs from a VM.

3. What is the difference between `RUN`, `CMD`, and `ENTRYPOINT`?

`RUN` executes while building. `ENTRYPOINT` sets the runtime executable. `CMD` provides the default command or arguments. Arguments after the image name replace `CMD`; `--entrypoint` replaces `ENTRYPOINT`.

4. Why copy `requirements.txt` before source code?

Docker can reuse the expensive dependency layer until the requirements input changes. If `COPY . .` comes first, any source edit invalidates that copy and every following install step.

5. Does `EXPOSE 8000` make an API available on the host?

No. It documents intended container ports. `docker run -p 8080:8000 ...` publishes host port 8080 to the container's port 8000.

6. Why must Uvicorn bind to `0.0.0.0`?

Binding to `127.0.0.1` listens only on the container loopback interface. `0.0.0.0` listens on all container interfaces, allowing traffic forwarded through Docker networking.

7. How should secrets be handled?

Inject runtime secrets using the platform's secret store or mounted secret files. For build-time credentials, use BuildKit secret/SSH mounts. Never commit them or bake them through Dockerfile `ARG`, `ENV`, `COPY`, or command history.

8. What are multi-stage builds?

They use multiple `FROM` stages. A builder contains compilers and headers; the final stage copies only wheels/binaries and runtime files. This reduces image size and attack surface.

9. How do you persist data?

Use named volumes, bind mounts, or external stores. Do not rely on the container writable layer because containers should be replaceable. Choose according to portability, performance, backup, and ownership requirements.

10. Why can `depends_on` still lead to startup failures?

Starting a process does not mean it is ready. Use a health check and `condition: service_healthy` where appropriate, but also implement client retry/backoff because a dependency can fail after startup.

11. How would you reduce a Python ML image?

Use an appropriate slim base, `.dockerignore`, multi-stage builds, wheels, `--no-cache-dir`, and copy only runtime artifacts. Avoid compilers, notebooks, data, tests, and duplicate model files in the final stage. Measure size and preserve required shared libraries.

12. How do containers access GPUs?

The host needs a GPU, compatible driver, and configured container GPU runtime/toolkit. The image supplies compatible user-space CUDA/framework libraries. The device is explicitly exposed, for example with `--gpus all` or a Compose device reservation.

13. Why might four API workers be worse than one?

Each process may load a separate multi-gigabyte CPU/GPU model, exhausting memory or causing GPU contention. Choose worker count through load tests; a single model process with batching may deliver higher throughput.

14. How do you make an ML image reproducible?

Pin dependencies and base digest, record platform, code/data/config/model versions, build in CI, test the image, store its digest and SBOM, and deploy that same digest. Seeds help model repeatability but cannot solve all nondeterminism.

15. Should model weights be inside the image?

It is a trade-off. Bundling gives an atomic, easy-to-roll-back unit but makes images and pulls large. External weights update independently but require checksum, provenance, compatibility, availability, and atomic version coordination.

16. How do you debug a container that exits immediately?

Check `docker ps -a`, exit code/state via `docker inspect`, and `docker logs`. Verify the main process/command, paths, permissions, environment, and artifact availability. Run an alternate shell only for diagnosis, then fix and rebuild the image.

17. What happens when a memory limit is exceeded?

The kernel may OOM-kill the container process, commonly producing exit code 137 (also possible after SIGKILL for other reasons). Inspect container state and host/runtime events; reduce memory, workers, batch/cache, or raise a justified limit.

18. What makes a good ML health check?

It is fast, deterministic, and distinguishes process health from model readiness. Readiness confirms the model is loaded and essential local state works; it should not run expensive inference or fail because of optional downstream dependencies.

19. How do tags and digests differ?

A tag is a mutable readable reference; a digest is derived from immutable image content. Use unique tags for humans and deploy/record digests for exact identity.

20. Does Docker prevent data leakage or model drift?

No. It freezes software/runtime, not experimental methodology or future data distributions. Leakage is prevented by correct splitting/pipelines; drift requires monitoring, ground truth, evaluation, and retraining policy.

17. Practice Tasks

1. **Small coding task:** containerize a FastAPI endpoint for a saved scikit-learn **Pipeline**; run non-root and add **/live** and **/ready**.
2. **Dataset project:** train an Adult-income classifier with a leakage-safe **ColumnTransformer**, serialize it, and serve typed requests.
3. **Experiment:** compare full vs slim and single-stage vs multi-stage builds. Record compressed image size, clean/cached build time, pull/startup time, and vulnerabilities.
4. **Load experiment:** vary workers and batch size; graph throughput, p95 latency, RAM, and GPU memory. Explain the saturation point with Little's Law.
5. **Debugging task:** intentionally bind to **127.0.0.1**, remove artifact permissions, and point **MODEL_PATH** incorrectly. Diagnose each using logs, inspect, exec, and health status.
6. **Security task:** prove a fake secret persists when copied into an earlier image layer, then replace it with a BuildKit secret mount. Never use a real credential.
7. **Reproducibility task:** rebuild with tag-only and digest-pinned bases; record image digests and explain sources of difference.
8. **Extension:** add Redis caching with Compose, service DNS, health-conditioned startup, timeouts, and runtime retry/backoff.
9. **GPU task:** verify **nvidia-smi**, framework CUDA availability, selected device, and memory use; document compatibility.

18. Project Ideas

Project 1: Reproducible tabular inference service

- **What it does:** trains a leakage-safe churn/fraud pipeline, publishes **/predict**, validates a strict schema, returns model version, and records latency/drift signals.
- **Tech stack:** Pandas, scikit-learn, FastAPI, Docker, pytest/stdlib smoke test, optional MLflow.
- **Dataset:** IBM Telco Customer Churn, UCI Adult, or Kaggle credit-card fraud with proper imbalance handling.
- **Resume value:** demonstrates end-to-end ML, preprocessing parity, API design, container security, testing, and observability.

Project 2: GPU text embedding and semantic search stack

- **What it does:** embeds documents, stores vectors, retrieves top-k passages, and exposes search with health/readiness and optional dynamic batching.
- **Tech stack:** PyTorch/Sentence Transformers, FastAPI, Docker GPU runtime, Qdrant/pgvector, Compose, Prometheus-compatible metrics.
- **Dataset:** BEIR subset, MS MARCO sample, or a documented public corpus.
- **Resume value:** shows GPU packaging, batching, vector retrieval, recall@k evaluation, multi-service networking, and capacity testing.

Project 3: Batch CV inference pipeline

- **What it does:** reads image references, performs versioned object detection/classification, writes predictions, supports restart-safe idempotent batches, and creates an evaluation report.
- **Tech stack:** PyTorch/torchvision or ONNX Runtime, Docker, object storage emulator, Python worker, Compose.
- **Dataset:** CIFAR-10 for classification or a small COCO/Open Images subset for detection.
- **Resume value:** demonstrates batch vs online trade-offs, artifact/version management, CPU/GPU images, mounts/object storage, failure handling, and cost measurement.

19. Quick Revision

Item	Recall
Key idea	Package code plus runtime dependencies as an immutable image; run it as replaceable containers
Main formula	$M_{worker} \approx M_{runtime} + M_{model} + BM_{sample} + M_{workspace}$
Use when	Environment consistency, deployment packaging, isolated jobs, reproducible CI
Operational metrics	p95/p99 latency, throughput, error rate, startup, CPU/RAM/GPU, OOM/restarts
Model metrics	Still task-specific; Docker does not replace offline/online ML evaluation
Common traps	latest , secrets in layers, root, bad cache order, no volume, wrong bind address, too many workers
Interview one-liner	"An image is an immutable layered package; a container is its isolated running process with a writable layer."

20. Final Cheat Sheet

Question	Answer
Definition	Docker builds, distributes, and runs container images
Input	Dockerfile, build context, base image, dependencies, code, optional model artifact
Output	Content-addressed image; at runtime, an isolated container process
Main steps	Ignore context → resolve base → cache/build layers → tag/push → run with config/mounts/network → health-check
Key controls	Base digest, platform, layer order, user, command, ports, mounts, environment, CPU/RAM/GPU, workers/batch
Quality metrics	Image size, build/pull/start time, vulnerability policy, p95/p99, throughput, errors, utilization
Pros	Portability, repeatability, isolation, immutable releases, cache/layer reuse, CI/CD fit

Question	Answer
Cons	Shared-kernel boundary, image/cold-start cost, state complexity, GPU/platform compatibility, operational learning curve
Best uses	Model APIs, batch/training jobs, CI, reproducible research environments, multi-service local development
Golden practices	Small context, pinned inputs, multi-stage build, non-root, no baked secrets, external state, cheap readiness, immutable digest

Command mini-sheet

```
docker build -t app:1.0 .           # Build
docker run --rm -p 8000:8000 app:1.0 # Run and publish port
docker ps -a                        # List including stopped
docker logs -f CONTAINER            # Follow logs
docker inspect CONTAINER            # Inspect state/config
docker exec -it CONTAINER /bin/sh   # Diagnostic shell
docker stats CONTAINER              # Resource usage
docker history app:1.0               # Layer history
docker compose up --build -d         # Start stack
docker compose down                  # Remove stack containers/network
```

Official references

- [Dockerfile and build best practices](#)
- [Build cache optimization](#)
- [Build secrets](#)
- [Compose startup order and health checks](#)
- [GPU access with Compose](#)